



임채현

BACKEND DEVELOPER

AI를 어디까지 쓸지 판단하는 개발자

문제를 먼저 정의하고, LLM에 맡길 부분과 코드로 확정할 부분의 경계를 나눈 뒤 구현합니다. 판단마다 기각한 대안과 트레이드오프를 남기고, 변경은 골든샷·회귀 테스트로 검증합니다.

EMAIL chaehyun010104@gmail.com
GITHUB <https://github.com/Chaehyunli>

PORTFOLIO <https://chaehyunlim-portfolio.vercel.app>
기술 블로그 <https://ch010104.tistory.com/>

강점과 근거

LLM·알고리즘 경계 설계

프롬프트로 지켜지지 않는 조건은 코드로 확정하고, 규칙으로 못 잡는 판단만 LLM에 남긴다.

모두약속 DeepEval 19개 테스트 검증 p.3

Masil 테스트 요청 24건 모두 정확히 분류 p.5

인증·신뢰 경계 설계

데이터가 저장되는 시점과 쓰이는 시점을 나눠, 서버가 무엇을 볼 수 있는지부터 정한다.

모두약속 서버도 키를 풀 수 없는 구조로 재설계 p.3

PETNER WebSocket 세션 인증 p.8

검색·파이프라인 최적화

병목을 연산과 통신, DB와 외부 호출로 나눠 측정한 뒤 그 구간만 고친다.

Searchive 왕복 5→1회 · 응답 250→10ms p.7

Masil k6 동시 접속 100명 실패 없이 처리 p.5

모두약속 네이버 검색 호출 36→21회 p.3

CAREER

2026.01 ~ 2026.02

프람트테크놀로지 · 개발팀 인턴 — 레거시 → MSA 전환 품질 검증

각자 맡은 모듈만 보던 방식에 의문을 갖고 레거시 코드를 공통 기준으로 약 20만 줄, 4개 모듈을 함께 검증했다.

8,000개

단위 테스트 설계

100%

인턴 기간 내 오류 해결률

p.2

SKILLS

LANGUAGES

Java · Python · TypeScript · SQL

BACKEND

Spring Boot · Spring Security · Spring WebFlux / R2DBC · FastAPI · JPA / SQLModel · WebSocket·STOMP · SSE

DATA · SEARCH

PostgreSQL · MySQL · Redis · Elasticsearch / OpenSearch · pgvector · Alembic / Flyway

AI 파이프라인

PydanticAI · DeepEval 골든샷·회귀 fixture · RAG · KeyBERT · XGBoost

INFRA

Docker Compose · Vercel · Render · Git

CAREER

2026.01 ~ 2026.02

Spring Boot

Thymeleaf

MyBatis

MSA

프람트테크놀로지 · 개발팀 인턴 — 레거시 → MSA 전환 품질 검증

약 15명이 약 20만 줄 규모의 전략물자 관리 시스템을 4개 모듈(portal·permission·judgement·destination) MSA로 옮기는 프로젝트에서 레거시 분석과 테스트 설계를 맡았다.

8,000개

단위 테스트 설계·수행

100%

인턴 기간 내 오류 해결률

4개

함께 검증한 MSA 모듈

모듈 단위 개발에서 전 모듈 검증으로

각자 맡은 모듈만 보는 방식으로는 모듈 간 표준을 세울 수 없고 요구사항 누락도 생긴다고 판단해, 레거시 코드를 공통 기준으로 모듈별 현황과 요구사항부터 정리했다.

필터 유지 요구를 방식 통일로 연결

검색 필터가 query 파라미터와 response body로 섞인 것을 발견했고, 관리원이 목록 복귀 시 필터 유지를 요청하자 조건이 URL에 남는 query 파라미터로 통일을 제안했다. 개발팀장 확인을 거쳐 전 모듈에 반영됐다.

오류는 원인 관점으로 공유

레거시와의 차이·재현 조건·발생 모듈 관점으로 정리해 공유했고, 찾은 오류는 인턴 기간 안에 모두 수정됐다.

EDUCATION & TRAINING

2021.03 ~ 2026.08

학력

명지대학교 컴퓨터공학부
컴퓨터공학과 졸업

2025.07 ~ 2025.08

교육

SK AI Dream Camp 중급

2026 ~ 현재

교육

SKALA 4기 — Agile/Scrum·MSA
팀 실습(SM), 응답 계약 기반 병렬 개발

자격

정보처리기사 · SQLD

PERSONAL AI

2026.07 ~ 현재

HERMES · 개인 프로젝트 · 설계·구축·운영 단독

대화로 정한 관심사·규칙을 기억해 뉴스·채용·학습 업무를 자동화하는 개인 AI 업무 환경

Hermes Agent · Linux VPS · Discord · MCP · Notion · Obsidian
· Cron · Python · Hook

92.1%

최근 30일 684건 중
630건 완료

10개

VPS에서 상시
운영하는 Cron 자동화

3개

LLM 없이 스크립트로
실행하는 자동화

AI 판단 오류를 규칙으로 보정

관심사·우선순위와 어긋날 때마다 대화 피드백을 메모리·Skill에 반영하고, 반복 오판은 Hook 규칙으로 전환했다.

스크립트와 AI의 역할 분리

규칙이 명확한 수정은 스크립트로 실행하고 요약·선별만 AI에 맡겼다. 원본 검증이 실패하면 Notion 쓰기를 멈추고 원인을 기록한다.

PROJECTS 4개 상세 · 2개는 웹 포트폴리오에서

2026.08 ~

p.3

모두약속

배포 운영 중

목적·시간·지역·예산·선호를 입력하면 실제 장소 검색 결과로 약속 일정을 만들어주는 AI 서비스

2026.03 ~ 2026.08

p.5

Masil

앱 배포 완료

대화로 일정을 짜고 예약까지 이어지는 AI agent 여행 플래너

2025.10 ~ 2025.12

p.7

Searchive

완성

문서를 업로드하면 자동 태깅·검색·RAG 질의응답으로 이어지는 개인 지식 베이스

2025.08 ~ 2025.10

p.8

PETNER

장려상

유기견 탐색·입양 신청·커뮤니티·보호소 실시간 채팅을 연결한 팀 백엔드 서비스

2025.01 ~ 2025.03

웹에서 보기 ↗

동아리모아

완성

동아리 탐색·지원·운영·권한 위임을 한 서비스에서 다룬 팀 백엔드 프로젝트

2025.03 ~ 2025.10

웹에서 보기 ↗

노소공

완성

행동 데이터로 펫 감정을 예측하고 미니게임 보상·성장 흐름을 연결한 ML 기반 동물 육성 게임

목적·시간·지역·예산·선호를 입력하면 실제 장소 검색 결과로 약속 일정을 만들어주는 AI 서비스

기간 · 형태

2026.08 ~ 개인 프로젝트

맡은 역할

기획·설계·구현·배포 단독

TECH

FastAPI SQLModel PostgreSQL Redis Vue 3 TypeScript Web Crypto API DeepEval

LINKS

배포 <https://moduyaksok.vercel.app>
 GITHUB <https://github.com/Chaehyunli/moduyaksok>
 상세 <https://chaehyunlim-portfolio.vercel.app/moduyaksok>

1개월

기획부터 배포까지 단독

실제 약속에 활용 중

19개

DeepEval 테스트로 검증

GEval 임계값 0.70

60만 회

PBKDF2 키 유도

서버도 키를 풀 수 없는 구조

일정을 만들기 위해 98개의 후보를 검색해보았어요

일정에 꼭 넣을 장소

대진 도원참치 강남역점

다시 일정 생성하기

다시 생성하면 위 장소를 모두 포함한 새 후보를 만들어요.

좋아한다고 말한 조건으로 찾은 장소

와플 · 9곳

초밥 · 10곳

싫어한다고 말해 일정에서 제외된 장소

햄버거 · 10곳

실제 검색한 장소 후보 - 일정은 이 안에서만 구성한다

배경

AI가 실재하지 않는 장소를 추천하거나 실제 이동거리를 반영하지 못해 일정이 틀어지는 한계가 있었다. 그래서 실제 검색 결과 안에서만 일정을 구성하고, 실제 경로로 동선을 검증하는 서비스를 혼자 한 달 만에 기획부터 배포까지 진행했다.

설계

AI 일정 생성 파이프라인 · LLM ↔ 알고리즘 경계

☒ LLM 호출
 ☐ 알고리즘 · 결정론적

STEP 1

LLM

조건 정규화

입력: 목적·인원·시간·지역·예산·선호
자유 텍스트 → 검증 가능한 태그 + 취향

STEP 1.5

외부 API

네이버 지역검색

카테고리 16종 × 태그만큼 팬아웃
실재하는 장소만으로 후보 풀 구성

STEP 2

LLM + 알고리즘

후보 생성 · 3관점 병렬

가성비 / 동선최소 / 취향반영
검색 풀 안에서만 선택 → 환각 차단

STEP 3-㉠

알고리즘 · 결정론적

규칙 기반 하드 필터

시간겹침·예산초과·이동거리·태그충복
좌표와 시간만으로 코드화 확정 판단

STEP 3-㉡

LLM

LLM 1회 호출

규칙을 통과한 후보만 대상으로
의미적 태그 위반 판단 + 추천 이유

STEP 4

알고리즘 · LLM 없음

동선 보강

선택된 후보에만 Odsay·NCP 호출
도보·대중교통·자차 경로 + 시간 보정

STEP 3-㉡ 이후 · 후보 3개를 제시하고, 사용자가 고른 1개에만 STEP 4를 실행해 외부 경로 API 호출을 아낀다

AI가 제안한 키 보관 구조를 검증해, 서버도 키를 풀 수 없게 재설계했다

문제	사용자가 자신의 LLM API 키를 등록하는 BYOK 구조다. 기획 기간을 줄이려 Manyfast AI로 초안을 잡고 Claude-GPT 답과 교차 검증하던 중, 초안의 서버가 마스터 키로 사용자 키를 복호화할 수 있어 서버 침해 시 등록된 키 전체가 노출될 수 있다는 위험을 발견했다.
검토한 대안	- 현업 개발자 자문: "편의성과 신뢰성 사이의 트레이드오프" — 이 기준으로 3가지 안을 검토 - 클라이언트에서 LLM 직접 호출 — 구현 비용이 큼 - 키를 저장하지 않고 매 요청마다 전달 — 요청마다 키가 평균으로 노출됨 - localStorage에 저장 — 브라우저를 닫아도 값이 남아 XSS 유출 위험
선택	- 암호화를 서버가 아닌 브라우저로: 사용자만 아는 비밀번호를 PBKDF2로 60만 회 반복해 만든 키로 AES-GCM 암호화, 서버에는 암호문·salt·iv만 저장 - Argon2는 WASM 의존성이 붙어 브라우저 내장 crypto.subtle의 PBKDF2 선택 - 비밀문구 원문은 키를 만든 직후 버리고, 만든 키만 탭을 닫으면 지워지는 sessionStorage에 두어 매번 입력하지 않게 하면서 XSS 노출 시간은 최소화
결과	서버도 사용자 키를 풀 수 없는 구조 로 바꾸고 비밀문구는 한 번만 입력 하게 해 신뢰성과 편의성의 균형을 맞췄다. 한 달 만에 배포해 실제 약속에 활용 중이다.
트레이드오프	비밀문구를 잃으면 키를 복구할 수 없어 기존 키 삭제 후 재등록으로 대응한다. sessionStorage도 XSS에서 자유롭지는 않아, 노출 대상과 시간을 줄이는 쪽을 택했다. provider 호출 순간에는 서버가 평균을 처리하므로 그 시점의 서버는 신뢰 경계 안에 있다.

LLM은 후보를 구성하고, 조건 보장은 알고리즘이 맡게 나눴다

문제	처음에는 일정 구성과 장소 선정을 모두 LLM에 맡겼다. 하지만 후보가 비슷한 장소로 수렴하거나 취향이 반영되지 않고 동선이 비현실적인 문제가 반복됐고, "태그당 후보 1곳", "식사 시간대엔 식사 장소" 같은 지시도 프롬프트만으로는 지켜지지 않았다.
판단 기준	- 시간 겹침·예산·이동거리·식사 슬롯·태그 중복은 좌표와 시각만으로 코드가 확정 가능 — 프롬프트로 부탁할 대상이 아님 "해산물 태그와 이자카야가 의미상 겹치는가" 같은 판단만 규칙으로 못 잡음 — 이것만 LLM에 남김
선택	- 알고리즘 기반 점수 필터링과 거리 제약을 단계적으로 추가 - 조건 정규화 → 후보 생성 → 검증·병합 → 동선 보강 4단계로 나누고, 검증 단계에서 규칙 하드 필터를 통과한 후보에만 LLM을 1회 호출 - LLM이 맡은 단계에는 DeepEval 테스트를, 코드가 맡은 조건에는 회귀 테스트를 두어 프롬프트·모델을 바꿀 때마다 같은 세트로 다시 검증
결과	수렴·취향·동선 세 가지 문제를 모두 해결 했고, DeepEval 19개 테스트 로 검증해 배포한 뒤 안정적으로 운영하고 있다.

외부 API 호출량 제어

요약	외부 장소 검색·경로 API의 호출량을 토큰버킷과 Redis 카운터로 제어했다. 예산이 부족하면 요청을 차단하는 대신 확보 가능한 만큼만 검색해 서비스가 멈추지 않게 했고, 하이브리드 전환으로 일정 1건당 네이버 검색 호출을 36회에서 21회 로 줄였다.
남은 과제	초당 리미터가 아직 in-process라 멀티 워커로 늘리면 전역 상한이 어긋난다.

대화로 일정을 짜고 예약까지 이어지는 AI agent 여행 플래너

기간·형태

2026.03 ~ 2026.08

4인 팀

맡은 역할

AI 파이프라인 설계 단독 · AI 서버 구현 · 백엔드 API 설계·일정 도메인 구현 · R2DBC 비동기 전환

팀원 4명 모두 AI 개발이 처음인 상황에서 자원해 에이전트 오케스트레이션을 독학하며 AI 서버를 맡았다.

TECH

React Native

Spring WebFlux

R2DBC

FastAPI

PydanticAI

PostgreSQL

Redis

SSE

DeepEval

LINKS

GITHUB <https://github.com/orgs/Masil2026/repositories>담당 PR [GitHub에서 담당 PR 목록 보기](#) ↗상세 <https://chaehyunlim-portfolio.vercel.app/masil>

24/24

테스트 요청 정확히 분류

DeepEval · 3개 시나리오 0.70~0.80

100명

동시 접속 실패 없이 처리

k6 · DB 커넥션 풀 15개 · 110명부터 지연

은상

명지대 캡스톤디자인 경진대회

26개 팀 중 · 원스토어 출시

항공편

확정

ICN

00:00

1시간 10분

CJU

01:10

2026.08.01 · Duffel Airways

₩60,000

AI 예약

예약번호: FLT-20260801-Q4W7KM

예약일 2026.05.20

배경

기존 ML 기반 여행 서비스는 일정만 제안할 뿐 실시간 항공·숙소·교통 정보를 반영하지 못했다. 대화로 일정부터 예약까지 처리하는 AI 에이전트를 기획하고, 예약까지 믿고 맡길 수 있는 에이전트를 목표로 삼았다.

예약 상태 - 잘못 취소되면 되돌릴 수 없는 화면

설계

AI Agent 파이프라인 · LLM ↔ 알고리즘 · 파란 배경 = 본인 영역



판단 01

되돌릴 수 없는 동작은 파이프라인 진입 전에 확정했다

문제	모호한 요청을 잘못 판단해 엉뚱한 예약이 실행됐다. "그거 취소해줘"가 일정 재생성인지 예약 취소인지 구분되지 않으면 되돌릴 수 없는 동작이 잘못 실행되고, 분류와 응답 생성이 한 단계에 섞여 있어 틀렸을 때 원인이 분류인지 계획인지도 구분할 수 없었다. 이대로 배포하면 사용자 신뢰를 잃는다고 판단했다.
판단 기준	• 분류를 파이프라인 안에서 하면 이미 실행이 시작된 뒤 — 진입 전에 판단해야 함 • 예약·취소처럼 되돌릴 수 없는 동작은 확정 전에 한 번 더 확인
선택	• classification_agent가 요청 의도를 5가지(대화·일정 생성·예약·수정·취소)로 먼저 분류 • 애매하면 실행하지 않고 다시 묻는 확인 단계로 분기 — 취소 대상이 불분명하면 후보를 먼저 제시 • 틀린 결과가 어느 단계에서 나왔는지 알 수 있게 분류·계획·합성 단계마다 DeepEval 평가를 붙임
결과	테스트 요청 24건을 모두 정확히 분류 하고 3개 시나리오 모두 GEval 0.70~0.80 을 확인한 뒤 원스토어에 출시했다. 26개 팀 중 은상 을 받았다.

판단 02

블로킹으로 남은 DB 접근을 비동기 전환과 호출량 제어로 풀었다

문제	WebFlux로 요청 처리를 비동기로 구성했지만 데이터 접근 계층은 블로킹 JPA였다. 응답이 느린 AI 호출이 몰리면 요청이 DB 커넥션을 쥔 채 기다려 적은 접속자에도 요청이 밀렸고, 외부 LLM API 호출이 몰리면 429 에러도 잦았다.
판단 기준	• WebFlux 위의 블로킹 JPA는 DB 구간에서 비동기 흐름을 끊음 — 느린 요청이 풀 크기만큼만 겹쳐도 다음 요청이 밀림 • R2DBC는 커넥션 수를 늘리는 게 아니라 커넥션을 붙잡는 시간을 줄임 — 같은 15개로 더 많은 요청을 처리 • 재시도만으로는 호출 폭주 자체를 못 막음 — 호출 빈도를 사전에 제어해야 함
선택	• 은상 수상 후 R2DBC를 직접 학습해 데이터 접근 계층까지 비동기로 전환하고, Reactive Redis·직렬화·테스트를 함께 정비 • Token Bucket으로 LLM 호출 빈도를 제어하고, 429 발생 시 지수 백오프로 재시도 • k6로 동시 접속을 단계적으로 늘리며 실패율과 p95 응답 시간을 기준으로 측정
결과	DB 커넥션 풀 15개 환경에서 동시 접속 100명까지 실패 없이 처리했고, 110명부터 지연 이 발생하는 한계선도 확인했다.
판단 근거	평균 대신 p95를 본 이유는, 평균은 느린 요청 몇 건을 가려 사용자가 실제로 겪는 지연을 놓치기 때문이다.

문서를 업로드하면 자동 태깅·검색·RAG 질의응답으로 이어지는 개인 지식 베이스

기간 · 형태

2025.10 ~ 2025.12

개인 프로젝트

맡은 역할

기획·설계·구현 단독

TECH

FastAPI

PostgreSQL

Elasticsearch

pgvector

Redis

MinIO

KeyBERT

React

LINKS

GITHUB

<https://github.com/orgs/Searchive-Project/repositories>

상세

<https://chaehyunlim-portfolio.vercel.app/searchive>

5→1회

Elasticsearch 네트워크 왕복

키워드 5개 기준

250→10ms

자동 태깅 응답 시간

96% 단축

3배

목표치 대비 태그 후보 과추출

필터 뒤 태그 0개 방지

배경

문서가 쌓일수록 필요한 자료를 찾기 어려워진다. 외부 서비스 없이 업로드부터 자동 태깅·검색·RAG 질의응답까지 이어지는 개인 지식베이스를 두 달간 혼자 기획·설계·구현했다.

sample.txt

2026/08/03 01:14 0 KB

Data Neural

Tensorflow

Deep Learning

업로드 직후 정제된 태그가 붙은 문서

설계

대표 태그 결정에서 검색·응답까지

정확한 일치부터 유사도 판단까지 순서대로 확인한 뒤, 검색 요청을 묶어 RAG 응답으로 연결

STEP 4

TagService · 대표 태그 결정

일치 여부를 우선순위로 평가

STEP 4-1 · 우선순위 1

이름 완전 일치

기존 대표 태그 재사용

STEP 4-2 · 우선순위 2

벡터 유사도 ≥ 0.8

기존 대표 태그로 수렴

STEP 4-3 · 마지막

둘 다 없음

새 대표 태그 생성

STEP 5

검색 최적화 · `_msearch`

N개 KNN 쿼리를 1회 요청으로 묶음

STEP 6 · LLM

RAG 질의응답

검색 결과를 근거로 답변 생성

계산과 통신이 함께 N배로 늘어나는 구조를 배치 검색으로 풀었다

문제	문서 키워드 후보를 기존 태그와 벡터 유사도로 비교해 재사용 여부를 판단하는데, 후보마다 Elasticsearch에 개별 요청을 보내 통신이 후보 수만큼 늘었다. 키워드 5개면 자동 태깅 응답이 250ms까지 늘어났다.
판단 기준	- 순차 방식과 배치 방식의 시간을 따로 측정 — 원인은 연산이 아니라 네트워크 왕복 횟수 - 후보를 하나의 쿼리로 합치면 후보별 결과가 섞임 — 검색 단위는 유지하고 전송만 묶어야 함
선택	- 후보별 KNN 쿼리는 그대로 두고 Elasticsearch _msearch 요청 한 번으로 묶어 보냄 - 응답을 원래 후보 순서로 복원해, 이후 완전 일치·유사도 판단이 기존과 같은 입력을 받게 함
결과	키워드 5개 기준 네트워크 왕복을 5회에서 1회 로 줄이고, 응답 시간을 250ms에서 10ms 로 96% 단축했다.

AI가 뽑은 태그 후보를 그대로 저장하지 않았다

문제	KeyBERT 후보에 불용어·숫자·한 글자 토큰이 섞여 필터링하면 태그가 0개가 되기도 했고, 같은 의미라도 표기가 다르면 별도 태그로 저장돼 하나의 개념이 여러 태그로 흩어졌다.
검토한 대안	- 목표 개수만 추출 — 필터링 뒤 후보가 모자라 품질 기준을 적용할 여지가 없음 - 완전 일치만 사용 — 표기 차이를 못 흡수 / 유사도만 사용 — 다른 의미를 합칠 위험
선택	- 태그 후보를 목표치의 3배로 과추출한 뒤 불용어·숫자·한 글자 토큰을 거르고 표기를 정규화 - 완전 일치 태그가 있으면 재사용, 없으면 pgvector 유사도 0.8 이상인 대표 태그로 병합, 둘 다 아닐 때만 신규 생성
결과	필터 뒤 태그가 0개가 되는 문제를 막고 , 표기 차이로 검색 필터가 분산되는 일을 줄여 사용자가 보는 태그를 일관되게 유지했다.

PETNER

장려상

유기견 탐색·입양 신청·커뮤니티·보호소 실시간 채팅을 연결한 팀 백엔드 서비스

기간·형태	2025.08 ~ 2025.10	4인 팀
맡은 역할	채팅·유기견·입양 신청·즐거찾기 도메인 백엔드 · API·ERD 단독 설계	
성과	명지대 2025 Cursor AI 기반 VIBE CODING 실전활용 경진대회 본선 8팀 중 장려상	
TECH	Java 17 Spring Boot Spring Security JPA PostgreSQL Redis OpenSearch WebSocket/STOMP	
LINKS	GITHUB https://github.com/orgs/Dangdaengdan/repositories 담당 PR GitHub에서 담당 PR 목록 보기 > 상세 https://chaehyunlim-portfolio.vercel.app/petner	

WebSocket 세션 인증

로그인 세션 인증이 REST와 WebSocket이라는 서로 다른 경계를 넘어도 끊기지 않도록 STOMP 핸드셰이크 구조를 설계하고, 채팅방 생성부터 이전 메시지 페이지까지 이 위에서 확장했다.

채팅방 나가기 = Soft Delete

참여 레코드를 비활성화해 FK 무결성을 지키고, 입장·퇴장 시각으로 메시지 범위를 나눠 재입장한 사람에겐 이후 메시지만, 보호소에겐 상담 이력이 남게 했다.

검색엔진 선택

오타 허용 검색이 필요해 Elasticsearch와 OpenSearch의 성능과 라이선스를 비교했고, 가볍고 Apache 2.0으로 제약이 적은 OpenSearch가 충분하다고 판단해 채택했다.